Exécution Symbolique

- Partie 1 -

~~Phantom Blood~~

Bases théoriques et CTFs

Rappels et motivation

Ça fait un moment que vous avez pas fait de reverse non ?

Statique Ou Dynamique ?

- Statique :
  - Approche précise
  - Peut être lent et complexe
- Dynamique :
  - Permet d’accélérer le reverse mais :
    - Mise en place d’un environnement d’analyse
    - Anti debug ?

- En pratique on aime faire les 2.

Statique Ou Dynamique – Again

- Statique :
  - Raisonnements manuels mais génériques.
    - Par exemple : On est dans cette branche pour tous les entier positifs
- Dynamique :
  - Facilement automatisable mais spécifique.
    - Par exemple : Si on rentre la valeur 1, on part sur telle branche.

- Et si.. On pouvait faire des analyses automatiques, ET génériques ?

Je vous mens !

- Exécution symbolique :
  - Pas (que) une façon de reverse un binaire.
  - On parle bien d’ANALYSE en général.

  - Initialement développé pour…
    - Trouver des bugs
    - Générer automatiquement des cas pathologiques qui mènent à ces bugs.

- Pour le reverse c’est génial aussi !

Les bases de l’exécution symbolique

Ça fait 10 ans que j’en parle, mais concrètement, c’est quoi ?

Les bases historiques

SELECT--A FORMAL SYSTEM FOR
TESTING AND DEBUGGING PROGRAMS
BY SYMBOLIC EXECUTION*

Robert S. Boyer
Bernard Elspas
Karl N. Levitt
Computer Science Group
Stanford Research Institute
Menlo Park, California 94025

1975

Programming Languages — B. Wegbreit Editor

Symbolic Execution and Program Testing

James C. King
IBM Thomas J. Watson Research Center

1976

Exécution concrète

- Instructions lues et interprétées.
  - C’est la « vraie » exécution.

- Chaque variable (e.g. un registre), a une valeur précise.
  - eax = 0x90909090
  - dword [0xcafebabe] = 0xefbeadde
  - …

Exécution Symbolique

- Symbole : Valeur non concrète.
  - Expression fonction de littéraux
- Etat symbolique :
  - ~ Ensemble des variables, un state counter, un path condition
    - Plusieurs façon de faire en fonction des cas…
- L’exécution est plus une **exploration**

hanced testing technique. Instead of executing a program on a set of sample inputs, a program is “symbolically” executed for a set of *classes* of inputs. That is, each symbolic execution result may be equivalent to a large number of normal test cases. These results can be checked

Un petit exemple pour la route

Sans Branchements

```
1   int unknown_func(int a, int b, int c) {
2       int x = a + b;
3       int y = b + c;
4       return x + y - b;
5   }
```

Un petit exemple pour la route

Sans Branchements

```c
int unknown_func(int a, int b, int c) {
    int x = a + b;
    int y = b + c;
    return x + y - b;
}
```

Soit α_1, α_2, α_3, des entiers sur 4 bytes.
Exécutons unknown_func(α_1, α_2, α_3)

SC: L2, PC: TRUE, {x = α_1 + α_2}

J’omet a, b et c pour la lisibilité

Un petit exemple pour la route

Sans Branchements

```c
int unknown_func(int a, int b, int c) {
    int x = a + b;
    int y = b + c;
    return x + y - b;
}
```

Soit α_1, α_2, α_3, des entiers sur 4 bytes.
Exécutons unknown_func(α_1, α_2, α_3)

SC: L3, PC: TRUE, {x = α_1 + α_2, y = α_2 + α_3}

Un petit exemple pour la route

Sans Branchements

```c
int unknown_func(int a, int b, int c) {
    int x = a + b;
    int y = b + c;
    return x + y - b;
}
```

Soit α_1, α_2, α_3, des entiers sur 4 bytes.
Exécutons unknown_func(α_1, α_2, α_3)

SC: L3, PC: TRUE, {x = α_1 + α_2, y = α_2 + α_3,
RV = α_1 + α_2 + α_2 + α_3 - α_2}

↓

RV = α_1 + α_2 + α_3

Utile dans certains cas…

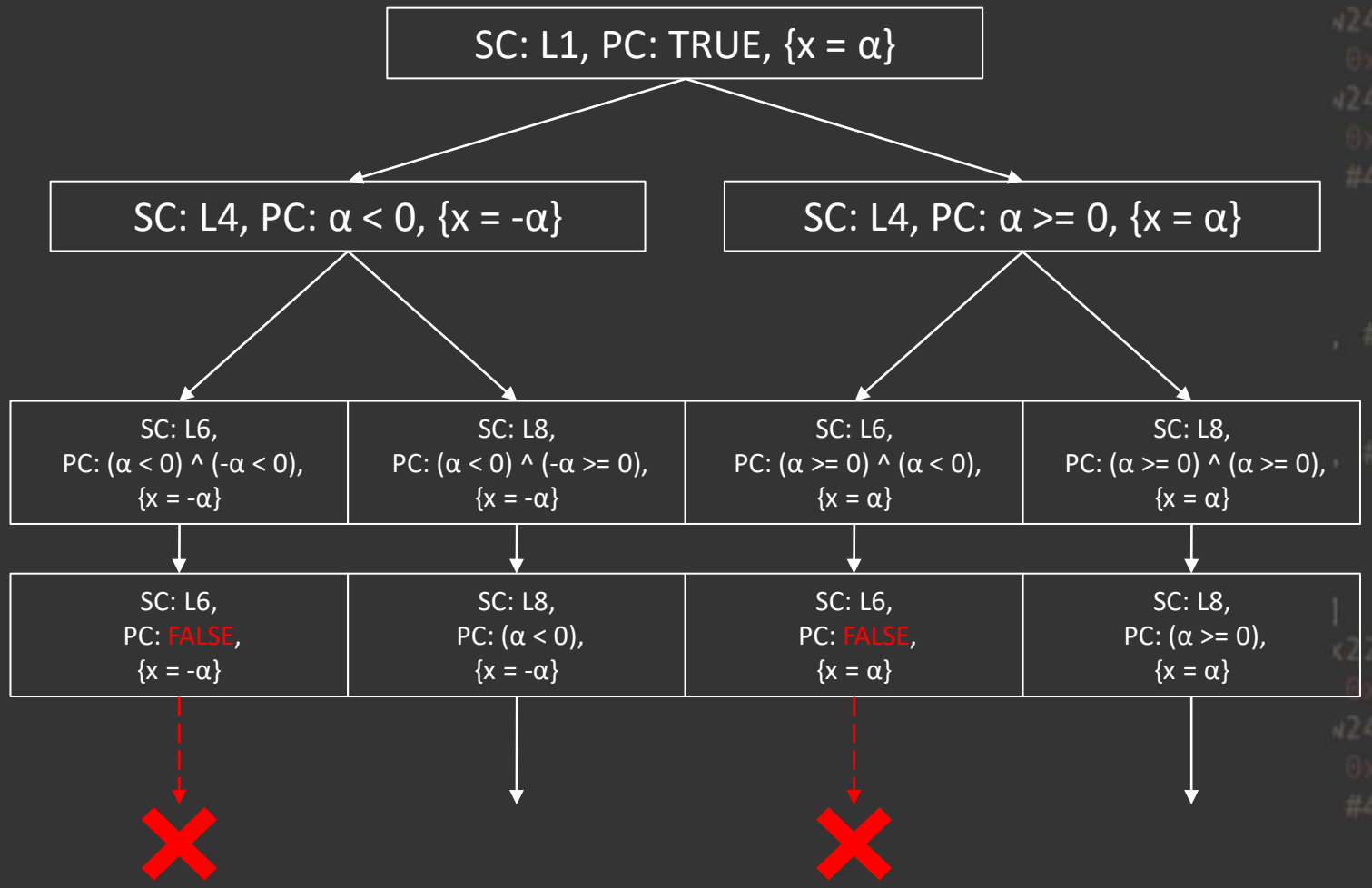
```
uint32_t unknown_func(uint32_t a, uint32_t b, uint32_t c) {
    return 3528490493 * (~((c | b) ^ c ^ a | c & c | c ^ b) ^ c) + 3092567603 * (((a | c) & b & a & (~a | b | b) | (b | a ^ a)
        & (a | c) & (a | c)) ^ a) + 3729467860 * (c ^ (~c & (b | a) & a & a | b & a & ((c | a) ^ b ^ c))) + 2399277776 * ~((c ^
        c | ~c) ^ c | (a & a | ~c) ^ ~c & (b | c)) + 2949053047 * ((~(a | a) | a ^ c & b) & ~(b & a & b) | ~(c ^ a ^ (a | b))
        & (b & b & ~c ^ ~a & a)) + 1877797717 * b + 3026634121 * (b & (~~c | (b | a) ^ (c | a) | ~(b & b | c | b))) + 3063147962
        * ~a + 1982891842 * ~(((a ^ b) & (c | c) | ~a | c) & ~(a & a) & (b ^ b | b ^ c)) + 651523277 * (c ^ ~b) + 2869011560 *
        (a & (c & c & b & c & ~~b | ~b ^ ~b ^ ~c ^ c)) + 1186361332 * (~~~~b & ((c | a) ^ a & a) & (b & b ^ c) & (c | a ^ a) &
        (a ^ b | c ^ b)) + 3286016414 * (((a ^ b | c ^ a) & ~~a | c) & (c ^ (~b | a | b | ~a ^ c ^ a))) + 2983746647 * (c ^ ~(a ^
        c) ^ (~~a | a) ^ (~b & b & a ^ (b | a ^ b)) & (~~b ^ ~b)) + 3938235111 * (a & b & (c | a)) + 2531773600 * (((b | c) ^
        ~a) & a & c & b ^ (b ^ c | b | a | a & a & b) | (~b | c | b | ~c ^ a) ^ (b ^ a | ~a)) + 3530434109 * (((a ^ c ^ b) & ~(a
        & c) ^ (b | c | b | c)) & ((~a | c ^ b) & ~a & b & a ^ a)) + 1329152113 * (((~c | a ^ b) & (b | b & b) ^ (c ^ c | a & a |
        ~(c | b))) & (~~(a & a) | b ^ b | ~a | ~a & c & b)) + 460285331 * (c | (~b | b | c) & (b & a | b ^ a) | (b & a ^
        ~b) & b | ~a) + 1979286536 * (~((a | c) ^ ~b) & a ^ ~(a ^ c | b ^ a) & ~a & (b | b) & (b ^ c)) + 1093134205 * (~~c &
        ((~a | a | a) ^ (a & a | a)) & b) + 2587807135 * (c ^ a ^ a ^ b & c | (a | c | b) & (a | a | b ^ b) | b) + 432032226 *
        ((a | ~~(a | a)) ^ ~b & ~(b & a & c)) + 1868996006 * ((b ^ a ^ b & a ^ ~b ^ b ^ b ^ ~(b ^ b ^ b)) & (a | (b | c | b) & (b
        | a) & (c ^ c))) + 1944964061 * (~(~b | b | ~(b & b)) | a) + 3435430685 * (~((c ^ b) & c) ^ (a ^ b ^ c ^ b) & (~b ^ b) ^
        (b & (b | a | a ^ b) | ~((a | a) ^ (a | a)))) + 1550252557 * ((~(b ^ a) | a & c ^ (a | b) | a ^ ~(c ^ a)) ^ a & (~c | c ^
        a) ^ b) + 393049221 * (b ^ (~~c | b & a & a & c) | ~~(c & b & a)) + 3290761574 * (~(a ^ a ^ c & b | b & c & (b ^ a)) &
        b) + 1196003411 * a + 735465009 * (b | a | a & a | c ^ a & c | a | a | c | b | (~c ^ (a | b)) & a ^ c) + 3147198221 *
        ~~((~c ^ a ^ c) & a) + 1115247542 * (~(b & a ^ b) & (c ^ c ^ b ^ c) & a ^ (~~b | ~(c & b) | c | b & a & b)) + 3198327722
        * (~((b | b) & (b | b)) ^ b | (~c ^ (c | b)) & a & (c ^ (b ^ a | b ^ a))) + 1754850977 * (((a ^ a ^ c & b | ~~b) ^ ~~a &
        ~a) & (~(c ^ b) & b | b & a | b & a | b)) + 2166274242 * (a & ~a) + 2770935394 * (~((a ^ c | b | a) & (c | c) & ~c) | (a
        ^ c) & ~c & b & b & c & a | a) + 3520732281 * (c | b) + 1905482535 * (b & (b ^ (b | c) ^ a ^ b) ^ ((b ^ a) & ~c ^ c)
        & ~~(c ^ b)) + 1199409103 * ~(~(a | c) | a | ~(a | c | a | b)) + 4049684919 * (~((a | c | a) ^ b) ^ b);
}
```

Comment on gère les if ?

- Le if est un fork de l’exécution
- On met à jour la PC pour prendre en compte la branche
- Si PC devient false, on peut stopper pour cette branche.

Exemple rapide

```c
void f(int x) {
    if (x < 0) {
        x = -x;
    }
    if (x < 0) {
        puts("Reachable ?");
    } else {
        puts("Reachable ?")
    }
}
```



L’instant questions

- Vous devez en avoir au moins 2
  - Dont la réponse est la même :)

La parenthèse z3

Y’a pas encore eu de forma à part entière…

Z3, c’est quoi

- Développé par Microsoft

- Prouveur de théorème.
  - Et donc on peut résoudre des équations/les simplifier
- Appelé pour résoudre les PC / trouver des inputs concrets qui mènent à un état.

Direct la pratique

- Z3 a des bindings python: `pip install z3-solver`
  - Si vous avez installé angr, vous avez déjà z3.

- Il suffit de faire des petits scripts.
  - `import z3` FTW !

Exemple trivial

```python
import z3

s = z3.Solver()
x = z3.Int("x")

s.add(x == 6)

if s.check() == z3.sat:
    print(s.model())
else:
    print("Unsat :(")
```

→ [x = 6]

Exemple facile

```python
import z3

s = z3.Solver()
x = z3.Int("x")

s.add(x**2 == 9)

while s.check() == z3.sat:
    m = s.model()
    print(f'x = {m[x]}')
    s.add(x != m[x])
```

x = -3

x = 3

Exemple avec 2 variables

```python
import z3

s = z3.Solver()
x = z3.Int("x")
y = z3.Int("y")

s.add(x*y == 6)
s.add(z3.And(x != 1, y != 1, x > 0))

while s.check() == z3.sat:
    m = s.model()
    print(f'x = {m[x]}')
    print(f'y = {m[y]}')
    print('---')
    s.add(z3.Or(x != m[x], y != m[y]))
```

→

x = 2
y = 3
---
x = 3
y = 2

Le TP impromptu

- Fin de forma = TP non ?
  - Et parenthèse = forma aussi…
- Le crackme4 du starhack est tout indiqué !

Fin de la parenthèse

On retourne sur la symbex

Le cas des boucles

- Parfois ça peut bien se passer
  - Le plus souvent obligé de fork à chaque tour…
    - C’est lourd
- Du coup on met des bornes (nb d’états, profondeur etc…)

Sur quoi on exécute ?

- Il faut formaliser la symbex sur un langage
- Souvent on défini un langage intermédiaire pour faire l’exécution
  - Angr : VEX IR
  - Miasm : Miasm IR
  - Radius2 : ESIL
  - …

Des exemples

- Angr : Moteur de base pour le joueur de CTFs
- Miasm : Avantage -> c’est bien plus que de la symbex, désavantage -> pas de doc
- Radius2 : One liners de folie, bindings en rust
- Manticore, KLEE, Binsec…

Les bases de Angr

Et comment ça peut nous aider pour les CTFs

On va voir quoi ?

- Les objets et les concepts importants
  - Vous allez voir, c’est proche de la théorie :)
- J’ai quasi tout extrait / adapté de la doc qui est excellente
- J’en ai fait un notebook IPython pour vous <3

Angr : Project

Permet de charger le binaire et de parser toutes les infos

```
p = angr.Project("ls")
print(p.arch)
print(f"Entrypoint : 0x{p.entry:X}")
print(p.filename)
```

✓ 0.3s

```
<Arch AMD64 (LE)>
Entrypoint : 0x405430
ls
```

Angr : Factory

Interface vers de nombreuses fonctionnalités

```
b = p.factory.block(p.entry)
print(b.pp())
```

✓ 0.0s

```
        _start:
405430  xor     ebp, ebp
405432  mov     r9, rdx
405435  pop     rsi
405436  mov     rdx, rsp
405439  and     rsp, 0xfffffffffffffff0
40543d  push    rax
40543e  push    rsp
40543f  lea     r8, [0x415c10]
405446  lea     rcx, [0x415ba0]
40544d  lea     rdi, [0x403bb0]
405454  call    qword ptr [0x61efc0]
None
```

Angr : State

- Ce sont des états symboliques !
  - Quand on a compris la théorie ça aide :).

```
state = p.factory.entry_state()
print(state.regs.rip)
print(state.regs.rip.concrete)

print(hex(state.solver.eval(state.regs.rip)))
print(hex(p.entry))
```

✓ 0.0s

```
<BV64 0x405430>
True
0x405430
0x405430
```

Angr : Simulation Manager

- Pour automatiser et simplifier l'exécution

```python
simgr = p.factory.simulation_manager(state)

print(simgr.active[0].regs.rip)
print(simgr.active[0].block().pp()) # Ends with Call [0x61efc0]
print(simgr.active[0].mem[0x61efc0].long.resolved) # Check Memory @ this address

simgr.step()

print(simgr.active[0].regs.rip) # It is indeed where we are going !
```

✓ 0.0s

```
<BV64 0x405430>
        _start:
405430  xor     ebp, ebp
405432  mov     r9, rdx
405435  pop     rsi
405436  mov     rdx, rsp
405439  and     rsp, 0xfffffffffffffff0
40543d  push    rax
40543e  push    rsp
40543f  lea     r8, [0x415c10]
405446  lea     rcx, [0x415ba0]
40544d  lea     rdi, [0x403bb0]
405454  call    qword ptr [0x61efc0]
None
<BV64 0x700010>
<BV64 0x700010>
```

```python
angr.loggers.disable_root_logger()

simgr = p.factory.simulation_manager(state)
for _ in range(19):
    simgr.step()

print(simgr.active[0].regs.rax.concrete) # At some point it becomes symbolic
print(simgr.active[0].regs.rax)
print(simgr.active[0].solver.eval(simgr.active[0].regs.rax))
```

✓ 0.3s

```
False
<BV64 unconstrained_ret_strrchr_1143_64{UNINITIALIZED}>
0
```

Le B-A BA pour un crackme

- La fonction « Explore »

```python
import angr
import claripy as clp
from binascii import unhexlify

angr.loggers.disable_root_logger()

p = angr.Project("crackme")
flag = clp.BVS("argv", 8 * 32)

init_state = p.factory.entry_state(args=["./crackme", flag])
simgr = p.factory.simulation_manager(init_state)

good = 0x400000 + 0x7d7
bad = 0x400000 + 0x7e5

simgr.explore(find = good, avoid = bad)

for state in simgr.found:
    print(unhexlify(f"{state.solver.eval(flag):X}"))
```

✓ 4.9s

```
b'MonSuperMotDePasse\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
```

Pour aller plus loin

- docs.angr.io
  - Bonne doc, avec un livre dédié à l’apprentissage.
- Si vous avez l’âme d’un pwner
  - https://blog.kylebot.net/2022/10/22/angry-FSROP/
- Un write up sympa :
  - https://care1es5.github.io/posts/IDA-Challenge-Free-Madame-de-Maintenon/

Petit TP

- Miaou, le chall du 120min CTF de 2022.

Des devoirs ?

- Pas eu le temps de faire un chall dédié,
- Mais vous pouvez faire :
  - https://github.com/jakespringer/angr_ctf

- La prochaine fois :
  - Exécution symbolique 2, Miasm et compagnie…